ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к решению полуфинального задания «Распознавание нестандартных государственных регистрационных знаков»

Дисциплина: «Искусственный интеллект и анализ данных»
Куратор: АИС Город
Дата: 25.10.2026

1. Цель и постановка задачи

Решение предназначено для полностью офлайн-распознавания российских государственных регистрационных знаков типов type1 (тип 1), type1a (тип 1А) и type1b (тип 1Б), а также для корректного отсева прочих знаков (other). На вход программе передаётся каталог изображений JPG/PNG. На выходе формируется CSV-файл с полями image;plate_num;plate_type;confidence. Во время распознавания решение не обращается к внешним API и может работать локально на ПК.

2. Данные и подготовка датасета

В комплект проекта включена воспроизводимая синтетическая часть датасета объёмом 5600 изображений. Генерация выполняется собственными скриптами с фиксируемым random seed. Для приближения к условиям реальной съёмки применяются перспективные искажения, размытие, шум, изменение яркости, JPEG-деградация и отрицательные примеры.

Класс	Количество
type1	1662
type1a	1729
type1b	1609
other	600
Итого	5600

Формат метаданных соответствует требованиям задания: для каждого знака могут фиксироваться относительный путь к изображению, plate_num, plate_type, bbox, четыре угловые точки quad, is_vehicle, is_synthetic, source, license и conditions. Для реальных изображений в проекте предусмотрен отдельный механизм сбора из открытых источников с проверкой лицензий. Автоматически предложенная разметка не выдаётся за ручную: её происхождение фиксируется в annotation_audit.csv, а неизвестные символы обозначаются символом #.

На момент формирования данной сборки локальный валидатор подтвердил корректность структуры синтетической части: Errors: 0. При этом он выводит предупреждения о том, что рекомендуемые объёмы реальных type1a, type1b и other ещё не набраны. Эти предупреждения не скрываются и должны учитываться при оценке ограничения текущей версии.

3. Архитектура решения

Пайплайн построен модульно и состоит из последовательных стадий:

- OpenCV формирует быстрые region proposals по геометрии и визуальным признакам.
- Локальный обученный SVM-фильтр detector_svm.joblib отсеивает ложные кандидаты.
- Найденный знак выравнивается перспективным преобразованием.
- Определяется тип знака type1/type1a/type1b/other; для целевых классов выполняется OCR.
- Основная OCR-модель ocr_svm.joblib использует HOG-признаки и LinearSVC; предусмотрен детерминированный fallback.
- Постобработка проверяет допустимый алфавит и маску российского ГРЗ, рассчитывает confidence и формирует CSV.

Разделение detect → rectify → classify → recognize → validate позволяет заменять отдельные компоненты более тяжёлыми моделями без изменения внешнего интерфейса программы. Веса моделей передаются вместе с исходным кодом.

4. Обучение и локальная валидация

В составе проекта находятся скрипты воспроизведения обучения. OCR SVM обучен на процедурно сгенерированных вариантах допустимых символов. На отдельной синтетической held-out выборке получена accuracy 1.0000 на 528 примерах. SVM-фильтр кандидатов на отдельной синтетической held-out выборке показал accuracy, precision, recall и F1 = 1.0000 на 720 примерах. Эти показатели характеризуют только процедурную синтетику и не интерпретируются как качество на скрытом тестовом наборе или на реальном видеопотоке.

Для воспроизводимости сериализованных SVM-моделей в requirements.txt зафиксирована версия scikit-learn==1.8.0, соответствующая версии, на которой были сохранены веса. Это устраняет предупреждение о несовместимости версий при загрузке моделей.

5. Производительность и инструкция запуска

Основная команда запуска решения на каталоге изображений:

```
python main.py --input <catalog> --output result.csv --benchmark
```

Контрольный запуск на локальном ПК участника 25.10.2026: обработано 3 изображения, получено 2 детекции, среднее время обработки составило 30,63 мс/изображение. Показатель ниже установленного в задании порога 100 мс/изображение, однако измерение выполнено не на референсном компьютере жюри и приводится только как локальный замер.

Для проверки сборки используются команды: python -m pytest -q; python dataset/validate_dataset.py --root dataset; python main.py --input test_images --output results/result.csv --benchmark. В текущей сборке модульные тесты проходят успешно: 2 passed.

6. Ограничения и дальнейшее развитие

Основное ограничение текущей версии связано с доменным разрывом между синтетическими и реальными изображениями. Качество может снижаться на очень мелких знаках, при сильном загрязнении, бликах, ночной съёмке, выраженном motion blur и сложном фоне. Кроме того, рекомендуемые объёмы реальных данных type1a/type1b/other в текущий архив не включены в полном объёме. Проект содержит механизм легального сбора и фиксации источников/лицензий, однако перед финальной конкурсной отправкой желателен визуальный аудит собранных реальных данных.

Архитектура допускает расширение на другие формы, виды и страны: для этого необходимо добавить новые классы детектора/классификатора, допустимый алфавит и соответствующие правила постобработки. Общий интерфейс запуска и формат выходного CSV при этом могут быть сохранены.